Demo: A Real-time Multi-Agent Network Attack Detection and Incident Response System

Arjun Sudheer*
Computer Engineering
San Jose State University
San Jose, USA
arjun.sudheer@sjsu.edu

Chia-Hong Chou*
Computer Science
San Jose State University
San Jose, USA
oliver.chou@sjsu.edu

Shubham Kumar
Software Engineer
San Jose, USA
shubhamk0894@gmail.com

2025 Silicon Valley Cybersecurity Conference (SVCC) | 979-8-3315-3429-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/SVCC65277.2025.11133641

***Abstract*—Network attacks have disrupted critical infrastructure and compromised important network operations worldwide for several decades. In spite of tremendous research outcomes based on various artificial intelligence techniques, current solutions still struggle to detect unknown network attacks and to adapt pretrained models in real time. This demo paper proposes a real-time multi-agent system to detect and react to the network dynamically and efficiently by using the LLM and RAG techniques. The proposed system consists of multiple agents: a data processing agent for feature engineering, a detection agent for traffic pattern analysis and attack detection, and a response agent for selecting appropriate response actions in the network in real time. Finally, based on RAG techniques, it provides a recommendation report for network administrators by searching databases of well-known vulnerability repositories, such as CVE. We demonstrate that the proposed system detects real-time network data with high accuracy. Its processing time is efficient enough to handle large volumes of network traffic.**

***Index Terms*—Large Language Model (LLM), Network Attack, Intrusion Detection System (IDS), Retrieval Augmented Generation (RAG), ReAct agent**

I. Introduction

A computer network attack involves using a computer network to disrupt, deny, degrade, or destroy information resident in computers and computer networks [1]. Such attacks can lead to brand damage, financial losses, and legal repercussions, in addition to the compromised data [2].

Current classifier-based, RNN-based, and anomaly detection methods, achieved high accuracy scores in detecting network traffic [3]–[5]. Although classifier, RNN-based, and anomaly detection methods have proven to be successful in detecting network attacks, they lack the ability to respond to such events and create a mitigation plan.

This paper makes the following contributions: First, it presents an LLM agent that uses domain knowledge to drop irrelevant network traffic features. Second, the proposed system integrates an ensemble classification method an agent for traffic classification. Third it provides a control agent that applies real-time mitigation via the Ryu controller. Finally, we integrate a recommendation agent that generates incident summaries and suggests relevant CVEs and security tools using retrieval-augmented generation (RAG).

[§]These authors contributed equally to this work.

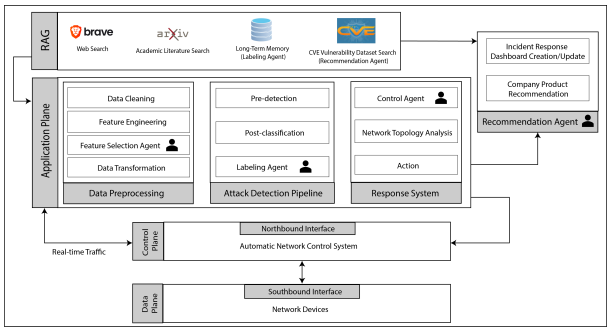


Fig. 1. System Architecture

Our real-time and scalable system achieves high accuracy (pre-detection: 0.993), f1-scores (post-classification: 0.981), and strong LLM relevance (0.78) and faithfulness (93.2%). This demonstrates the effectiveness of integrating LLMs with computer networks for intelligent, real-time network defense.

II. Background

Large Language Models (LLMs) have demonstrated strong capabilities in contextual understanding, summarization, and analysis tasks [6]. While AI-based firewalls and intrusion detection systems offer high classification accuracy, they often lack interpretability [7]. LLMs address this limitation in Explainable AI (XAI) by producing human-readable outputs [8]. Frameworks such as LangChain [9] implement the ReAct paradigm [10], enabling LLM agents to perform chain-of-thought reasoning, use tools, and execute function calls autonomously. These agents have shown potential in anomaly detection and network traffic pattern recognition [11]. Building on these foundations, we explore LLM agents for real-time network response, aligning with approaches like Li et al.'s IDS-Agent, which outperforms traditional methods in both classification accuracy and interpretability [12].

III. Methodology

A. System Architecture

Figure 1 presents a system architecture consisting of four key components: data preprocessing, an attack detection pipeline, a response system, and a recommendation agent. The

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on March 11,2026 at 19:46:21 UTC from IEEE Xplore. Restrictions apply.

system processes incoming network traffic through a multi-stage pipeline. The preprocessing module formats traffic data for analysis, followed by the attack detection pipeline, which classifies traffic as malicious or benign. The response system, built on SDN using Mininet and the Ryu controller, applies real-time traffic control via OpenFlow switches based on LLM-driven alerts—blocking, rerouting, or throttling threats. The recommendation agent supports incident response by generating reports and suggesting relevant security products for mitigation.

B. Dataset

We use the ACI IoT Network Dataset 2023 [13] provided by the United States Army. The dataset contains PCAP files from benign and malicious network traffic which are extracted into CSV format for use with our classifiers and LLM agents. Initially, the dataset is 65 MB with 84 features. After preprocessing, it is reduced to 20 MB with 16 features, 13 of which are engineered.

C. Data Preprocessing

We preprocess the dataset by removing duplicates, invalid values, constant features, and infrequent labels to improve generalization and enable stratified K-fold validation. Time-based features over one-second windows are engineered to enhance post-classification performance. The feature selection agent builds on the agent proposed by Jeong et al., which surpasses traditional methods [14]. Categorical features are encoded with OneHotEncoder [15], scaled using StandardScaler [16], and labels are transformed with LabelEncoder [17].

D. Attack Detection Pipeline

To handle unbalanced datasets, we train the classifiers using Stratified ten-fold Cross Validation [18]. We also use Optuna [19] for hyperparamter optimizations.

Pre-detection: The pre-detection stage employs a Random Forest classifier [20] for its fast inference speed, labeling incoming traffic as either benign or malicious. Benign traffic passes through the system, while malicious traffic proceeds to the post-classification stage.

Post-classification: We use four multi-class classifiers to identify the specific type of attack in real time: Random Forest [21], XGBoost [22], Multi-Layer Perceptron (MLP) [23], and K-Nearest Neighbors (KNN) [21]. The final prediction is determined via majority voting among the four classifiers and is passed to the response system. If no majority is achieved, the traffic is forwarded to the labeling agent.

Labeling Agent: The labeling agent provides the final attack class in cases of classifier disagreement. Following a method similar to Li et al. [18], we pass the packet information to the LLM-based labeling agent for final classification.

E. Response System

Controller Agent: Makes decisions based on classification results and network topology to determine the appropriate action.

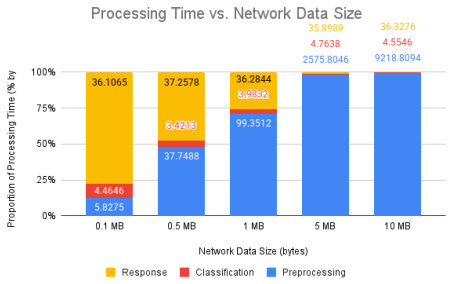


Fig. 2. Post-classification Majority Voting Precision-Recall Curve

Actions: Includes sending alerts, executing Linux commands on affected hosts, and reporting results. Actions use Retrieval-Augmented Generation (RAG) [24], combining prior detections with academic and real-time web knowledge on attack mitigation.

F. Recommendation Agent

The recommendation agent generates a comprehensive, human-readable dashboard for network incident response. It leverages API capabilities to retrieve CVE entries from the National Vulnerability Database (NVD) [25], historical attacks from a Long-Term Memory database, related research through the ArXiv API [26], and relevant product recommendations via web search.

IV. Results and Analysis

A. Processing Time Performance

Figure 2 shows processing times for preprocessing, attack detection, and response across different network data sizes. Classification and response times remain constant since they handle one packet at a time, while preprocessing time grows with larger data due to more samples in feature engineering.

B. Attack Detection

The pre-detection classifier achieved an accuracy of 0.993, while the post-detection ensemble achieved a precision, recall, and F1-score of 0.981. Figure 3 presents the precision-recall curve of the majority-voting classifier in the post-detection stage, demonstrating consistently high precision and recall across all attack classes. A slight decline in performance is observed for UDP Flood samples.

C. Response System & Recommendation Agent

To evaluate the output quality of the Retrieval-Augmented Generation (RAG) system, we assess two key metrics:

a) Faithfulness: Faithfulness quantifies the degree to which the LLM-generated output (*hypothesis*) is supported by the retrieved context (*premise*). Following the methodology of Maynez et al. [27], we compute faithfulness using a Natural Language Inference (NLI) model (DeBERTa-v3-base-mnli-fever-anli [28]):

$$\text{Faithfulness} = P(\text{entailment} = \text{LLM output} \mid \text{premise}, \text{hypothesis} = \text{retrieved context}) \quad (1)$$

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on March 11,2026 at 19:46:21 UTC from IEEE Xplore. Restrictions apply.

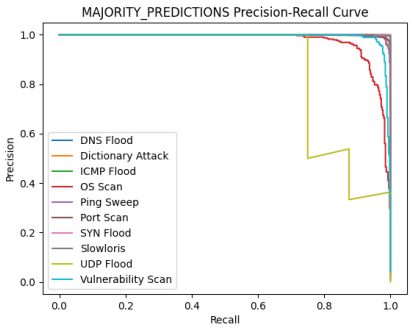


Fig. 3. Post-classification Majority Voting Precision-Recall Curve

b) Relevance: Relevance measures the semantic similarity between the cleaned retrieved context and the cleaned LLM output. Following the methodology of Lewis et al. [24], we compute relevance as the cosine similarity between their embeddings, using the nomic-embed-text model [29]:

$$\text{Relevance} = \cos(\mathbf{e}_{\text{context}},\ \mathbf{e}_{\text{output}}) = \frac{\mathbf{e}_{\text{context}} \cdot \mathbf{e}_{\text{output}}}{\|\mathbf{e}_{\text{context}}\| \cdot \|\mathbf{e}_{\text{output}}\|} \tag{2}$$

Here, $\mathbf{e}_{\text{context}}$ and $\mathbf{e}_{\text{output}}$ are the embedding vectors of the cleaned context and output, respectively. Based on an evaluation of 1% of the test set, the average faithfulness of the LLM outputs is **93.2%**, while the average relevance is **0.78**.

V. Conclusion and Future Work

This paper presents a network response framework that integrates LLMs with RAG capabilities to address key limitations of existing intrusion detection and prevention systems. As future work, we aim to address pre-detection misclassifications and further reduce operational overload.

Acknowledgment

This work is part of the SVCC 2025 CyberWarrior LLM Challenge. The project was conducted under Dr. Younghee Park's supervision.

References

[1] C. C. Editor, "Computer Network Attack (CNA) - Glossary: CSRC," https://csrc.nist.gov/glossary/term/computer_network_attack, cSRC Content Editor, accessed Apr. 9, 2025.

[2] Friendly Captcha, "What is network-based (cyber) security?" https://friendlycaptcha.com/wiki/what-is-network-based-cyber-security/#:~:text=A%20breach%20in%20network%20security,network%20security%20breach%20also%20increases, accessed Apr. 9, 2025.

[3] H. Zhang, Y. Li, Z. Lv, A. K. Sangaiah, and T. Huang, "A real-time and ubiquitous network attack detection based on deep belief network and support vector machine," *IEEE/CAA Journal of Automatica Sinica*, vol. 7, no. 3, pp. 790–799, May 2020.

[4] Y. Fu, F. Lou, F. Meng, Z. Tian, H. Zhang, and F. Jiang, "An Intelligent Network Attack Detection Method Based on RNN," in *2018 IEEE Third International Conference on Data Science in Cyberspace (DSC)*, Guangzhou, China, 2018, pp. 483–489.

[5] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network Anomaly Detection: Methods, Systems and Tools," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 303–336, 2014, first Quarter 2014.

[6] IBM, "What Are Large Language Models (LLMs)?" https://www.ibm.com/think/topics/large-language-models, accessed April 13, 2025.

[7] S. Ahmadi, "Next generation ai-based firewalls: A comparative study," *International Journal of Computer*, vol. 49, no. 1, pp. 245–262, 2023. [Online]. Available: https://ijcjournal.org/index.php/InternationalJournalOfComputer/article/view/2168

[8] A. Bilal, D. Ebert, and B. Lin, "Llms for explainable ai: A comprehensive survey," *arXiv preprint arXiv:2504.00125*, 2025. [Online]. Available: https://arxiv.org/abs/2504.00125

[9] LangChain Project Contributors, "LangChain," https://github.com/hwchase17/langchain, accessed Apr. 13, 2025.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[11] R. Molleti, V. Goje, P. Luthra, and P. Raghavan, "Automated threat detection and response using LLM agents," *World Journal of Advanced Research and Reviews*, vol. 24, no. 2, pp. 079–090, Nov. 2024.

[12] Y. Li, Z. Xiang, N. D. Bastian, D. Song, and B. Li, "Ids-agent: An LLM agent for explainable intrusion detection in IOT..." https://openreview.net/forum?id=iiK0pRyLkw, openReview, accessed Apr. 12, 2025.

[13] Army Cyber Institute, "ACI IOT Network Traffic Dataset 2023," https://cyber.army.mil/Research/Research-Labs/Datasets/#dataset1, accessed Apr. 11, 2025.

[14] D. P. Jeong, Z. C. Lipton, and P. Ravikumar, "LLM-select: Feature selection with large language models," https://arxiv.org/abs/2407.02694, arXiv.org, accessed Apr. 11, 2025.

[15] scikit-learn, "OneHotEncoder," https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html, accessed Apr. 11, 2025.

[16] ——, "Standardscaler," https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html, accessed Apr. 11, 2025.

[17] ——, "LabelEncoder," https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html, accessed Apr. 11, 2025.

[18] ——, "Stratifiedkfold," https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html, accessed Apr. 11, 2025.

[19] Optuna, "A hyperparameter optimization framework," https://optuna.readthedocs.io/en/stable/, accessed Apr. 11, 2025.

[20] scikit-learn, "Randomforestclassifier," https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html, accessed Apr. 11, 2025.

[21] XGBoost Documentation, "XGBoost documentation," https://xgboost.readthedocs.io/en/release_3.0.0/, accessed Apr. 11, 2025.

[22] pytorch.org, "Building Models with PyTorch — PyTorch Tutorials 2.0.0+Cu117 Documentation," https://pytorch.org/tutorials/beginner/introyt/modelsyt/-tutorial.html, 30 Nov. 2021, accessed Apr. 11, 2025.

[23] scikit-learn, "Kneighborsclassifier," https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html, accessed Apr. 11, 2025.

[24] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.

[25] N. I. of Standards and Technology, "National vulnerability database (nvd)," https://nvd.nist.gov/, accessed: 2025-04-13.

[26] arXiv, "arxiv api access," https://info.arxiv.org/help/api/index.html, accessed: 2025-04-13.

[27] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," *arXiv preprint arXiv:2005.00661*, 2020.

[28] M. Laurer, W. van Atteveldt, A. Salleras Casas, and K. Welbers, "Less annotating, more classifying – addressing the data scarcity issue of supervised machine learning with deep transfer learning and bert - nli," https://osf.io/74b8k, June 2022, preprint.

[29] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," 2024.

VI. Appendix

Our demo video is available at this link: https://youtu.be/CRR4-pxZ01Q. Our source code is available at this link: https://github.com/arjunsudheer/cyberwarrior-llm-challenge.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on March 11,2026 at 19:46:21 UTC from IEEE Xplore. Restrictions apply.